

RockyBot — News Research Assistant

Retrieval-Augmented Generation (RAG) application for question-answering over news articles

- Python
- LangChain
- Google Gemini
- FAISS
- Streamlit
- RAG / NLP

1. Executive Summary

RockyBot is a Retrieval-Augmented Generation (RAG) application that answers natural-language questions about news articles. A user supplies one or more article URLs through a Streamlit interface; the application extracts and indexes the article content as vector embeddings, and answers subsequent questions by retrieving the most relevant passages and generating a grounded response using a Google Gemini language model. Each answer is returned together with the source URL it was derived from. The project is a portfolio-level implementation built to demonstrate practical, end-to-end understanding of the RAG pattern — from document ingestion through retrieval to grounded generation — rather than a production deployment.

2. Project Overview

The application accepts up to three news article URLs, loads and processes their content, and builds a searchable vector index from it. A user can then ask questions in plain English and receive answers generated specifically from the indexed articles, along with the URL used as the source.

Processing pipeline:

1. Accept news article URLs through a Streamlit interface
2. Load the article content from each URL
3. Split the article text into smaller chunks
4. Convert the chunks into numerical vector embeddings using Gemini Embeddings
5. Store the embeddings in a FAISS vector index
5. Convert the user's question into an embedding
7. Retrieve semantically relevant document chunks from FAISS
3. Send the retrieved context to the Gemini LLM
3. Generate a context-grounded answer
3. Display the answer along with the source URL

Why not just ask the LLM directly? An LLM queried without supplied context can only answer from its training data — it has no knowledge of a specific, recently published article, and pasting full articles into every prompt is both token-expensive and limited by context-length constraints. RAG solves this by retrieving only the passages relevant to the question and grounding the answer in that retrieved text, which reduces cost, keeps the answer current, and ties the response to a verifiable source.

3. Business Problem

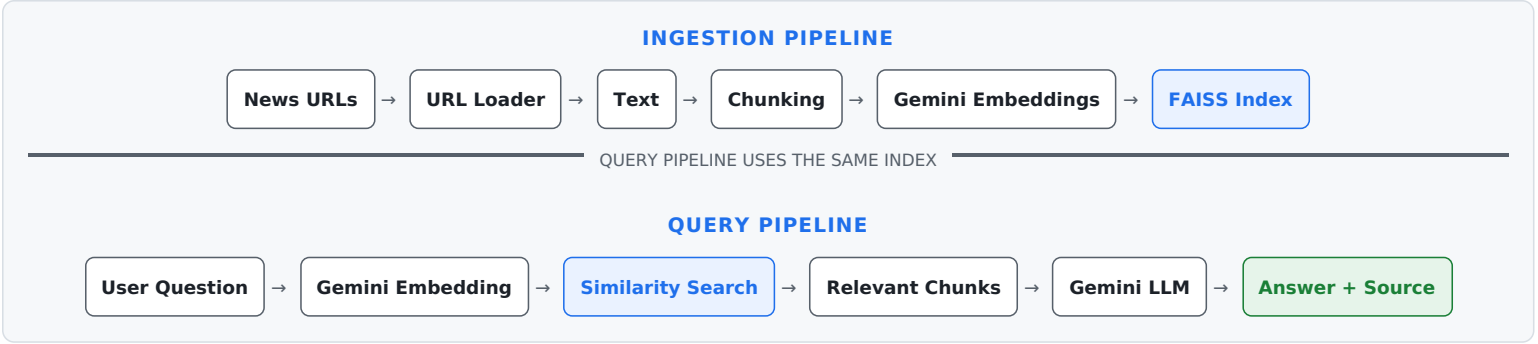
Extracting specific information from news articles is a time-consuming, manual process: a reader has to locate the relevant article, read through it, and pick out the fact or figure they need. This becomes harder as the number of articles grows or when the answer to a question is spread across more than one source.

This is fundamentally an **information retrieval and knowledge extraction problem**. RockyBot addresses it by giving users a conversational research interface where they can:

- Provide article URLs as a knowledge source
- Ask direct questions ("What is the price of X?")
- Request summaries
- Identify important facts or features
- Retrieve specific figures or details
- Receive answers grounded in the provided articles, with the source cited

4. Solution

RockyBot solves the problem using a RAG architecture, which separates the process into an **ingestion/indexing** stage (run once per set of URLs) and a **query/retrieval/generation** stage (run per question). At a high level:



5. Objectives

- Build a working end-to-end RAG pipeline over user-supplied news URLs
- Use Google Gemini for both embeddings and answer generation
- Use FAISS for efficient local vector similarity search
- Provide a simple, functional Streamlit interface for non-technical use
- Ensure every generated answer is traceable back to a source URL
- Persist the vector index locally so it does not need to be rebuilt on every application start

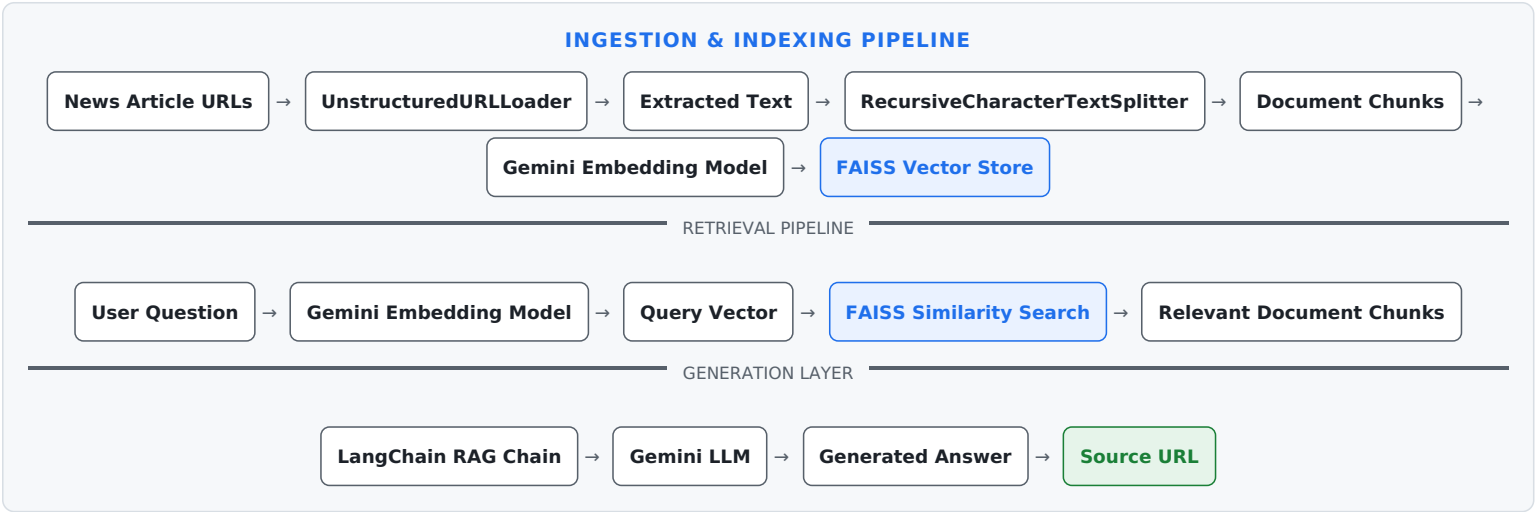
6. Technology Stack

Layer	Technology
Frontend / UI	Streamlit
LLM	Google Gemini — <code>gemini-3.5-flash-lite</code>
Embedding Model	Gemini Embeddings — <code>models/gemini-embedding-2</code>
Orchestration Framework	LangChain
Vector Database / Similarity Search	FAISS
Document Loading	<code>UnstructuredURLLoader</code>
Text Splitting	<code>RecursiveCharacterTextSplitter</code>
Programming Language	Python
Local Persistence	FAISS <code>save_local()</code> / <code>load_local()</code>
API Authentication	Gemini API key, stored in a separate local <code>secret_key.py</code>

Note: the original tutorial this project was adapted from used OpenAI as the LLM and embedding provider. This implementation replaces both with Google Gemini throughout.

7. System Architecture

The system is organized into three clearly separated layers: ingestion/indexing, retrieval, and generation.



8. RAG Architecture — Concept Explanation

Question	Answer
What is RAG?	Retrieval-Augmented Generation: instead of relying only on what an LLM learned during training, the system first retrieves relevant text from an external source and supplies it as context before the LLM generates its answer.
Why is retrieval required?	The LLM has no built-in knowledge of the specific articles supplied by the user, and cannot be re-trained per query. Retrieval supplies exactly the information needed at answer time.
Why split documents into chunks?	Full articles can exceed the model's usable context and increase cost. Smaller chunks let the system retrieve only the passages relevant to a given question.
What are embeddings?	Numerical vector representations of text that capture semantic meaning rather than exact wording.
Why numerical vectors?	Vectors allow meaning to be compared mathematically (via distance/similarity) rather than through exact keyword matching.
What does FAISS do?	Stores the vector embeddings and performs fast similarity search to find the chunks closest in meaning to a query.
Why similarity search?	The most semantically relevant chunks are the ones most likely to contain the answer, even if they don't share the exact wording of the question.
What happens on a question?	The question is embedded, FAISS retrieves the closest chunks, and those chunks are passed to Gemini as context for the final answer.
What role does Gemini play?	Interprets the question together with the retrieved context and generates the natural-language answer — it does not perform the search itself.

Clear separation of responsibilities:

FAISS = retrieval / similarity search only.

Gemini (LLM) = reasoning and answer generation only.

LangChain = orchestration layer connecting retriever, context, and LLM.

Streamlit = user interface only.

9. Document Ingestion Pipeline

Ingestion follows this sequence:

```
URL → UnstructuredURLLoader → Article text → RecursiveCharacterTextSplitter → chunks → Gemini embeddings → FAISS
```

`UnstructuredURLLoader` fetches and parses the HTML of each supplied URL into plain article text. That text is then split into chunks using `RecursiveCharacterTextSplitter` before being embedded and indexed. No additional preprocessing steps beyond loading and chunking were implemented.

10. Text Chunking

Chunking is performed with `RecursiveCharacterTextSplitter` , using approximately:

```
chunk_size = 1000
chunk_overlap = 200
```

The splitter tries a sequence of separators (paragraph breaks, line breaks, sentence boundaries) so chunks stay close to the target size without cutting words or sentences awkwardly. The 200-character overlap between consecutive chunks ensures that context spanning a chunk boundary — for example, a sentence that continues an idea from the previous paragraph — is not lost, which reduces the risk of retrieving a chunk that is missing the information needed to answer a question fully.

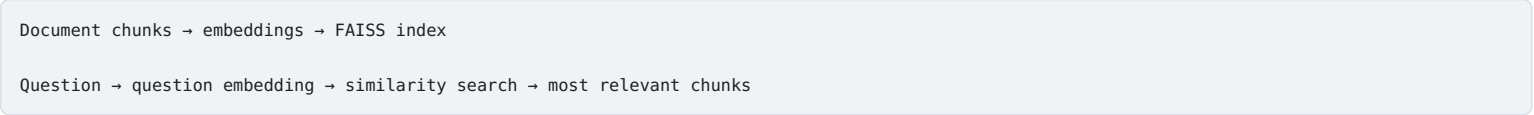
11. Gemini Embeddings

Embeddings are generated using `models/gemini-embedding-2` . This model converts a piece of text into a numerical vector that captures its semantic meaning, rather than just its literal wording.

Conceptual example: the sentences "*Tesla launched a new AI initiative*" and "*Tesla announced an artificial intelligence project*" use different words but express a similar meaning. Their embedding vectors would be close together in vector space, which is what allows the system to retrieve a relevant chunk even when the user's question doesn't share the article's exact vocabulary — this is **semantic retrieval**, as opposed to simple keyword matching.

12. FAISS Vector Store

FAISS (Facebook AI Similarity Search) is used to efficiently store and search the vector embeddings generated from the article chunks.



The FAISS index is persisted locally so that an existing index can be loaded without rebuilding it, as long as the indexed document set remains unchanged:

```
# Saving the index
vectorindex_gemini.save_local("vector_index")

# Loading the index
FAISS.load_local(
    "vector_index",
    embeddings,
    allow_dangerous_deserialization=True
)
```

13. Query Processing

When a user submits a question, it is converted into an embedding using the same Gemini embedding model used for the article chunks. FAISS then performs a similarity search between the question vector and the stored chunk vectors, returning the chunks that are semantically closest to the question. Only these retrieved chunks — not the full article set — are passed forward as context for answer generation.

14. Gemini as the Large Language Model

The original educational tutorial this project is based on used OpenAI as the LLM. This implementation was adapted to use Google Gemini instead, specifically `gemini-3.5-flash-lite`.

Gemini's role in the pipeline is strictly generation, not retrieval:

- Receives the retrieved context chunks from FAISS
- Interprets the user's question in light of that context
- Generates the natural-language answer
- Produces a response grounded in the retrieved article content, rather than in its own general training knowledge

Component	Responsibility
Gemini Embeddings	Vector representation of text
FAISS	Similarity retrieval
Gemini LLM	Answer generation

15. LangChain Orchestration

LangChain acts as the orchestration layer connecting the individual components. The application uses `RetrievalQAWithSourcesChain`, which ties together:

- The FAISS retriever
- The retrieved context chunks
- The Gemini LLM
- The final generated answer
- The source metadata (URL) associated with the retrieved chunks

This chain is what allows the application to return both an answer and its source URL from a single call, rather than requiring separate logic to track provenance.

16. Streamlit Application

The interface is intentionally simple:

Area	Elements
Sidebar	URL 1, URL 2, URL 3 input fields; "Process URLs" button
Main area	Question input box; Answer section; Sources section

17. End-to-End Workflow

- 1. User enters article URLs in the sidebar
- 2. User clicks "Process URLs"
- 3. Articles are loaded and processed via `UnstructuredURLLoader`
- 4. Article text is chunked and Gemini embeddings are generated
- 5. A FAISS vector index is created and stored locally
- 6. User enters a question in the main input box
- 7. Relevant content is retrieved from the FAISS index
- 3. Gemini generates the answer from the retrieved context
- 3. The answer and its source URL are displayed to the user

18. Example Execution

One of the tested queries was:

"Summarize the Punch iCNG article in 3 points."

The application returned a three-point, source-grounded summary:

- 1. Tata Motors launched the Punch iCNG and its price range
- 2. The vehicle uses twin-cylinder technology and includes safety mechanisms
- 3. The vehicle includes features such as an electric sunroof, projector headlamps, LED DRLs, alloy wheels, infotainment, rain-sensing wipers, and a height-adjustable driver's seat

Along with the summary, the application returned the corresponding Moneycontrol source URL, demonstrating that the system successfully retrieved relevant article content and produced a source-grounded answer rather than a generic response from the model's general knowledge.

19. Validation

The system was tested with a range of question types against the indexed article, including:

- "What is the price of the Punch iCNG?"
- "What are the main features of Punch iCNG?"
- "What safety features does Punch iCNG have?"
- "Summarize the Punch iCNG article in 3 points."

In each case, the answer was generated from the indexed article content, with the corresponding source URL displayed alongside it. No formal accuracy metrics were measured; validation was functional — confirming that answers were grounded in the correct source article.

20. Implementation Challenges & Solutions

The following engineering issues were encountered and resolved while adapting the original OpenAI-based tutorial to Gemini:

Challenge	Resolution
The tutorial's embedding model, <code>models/text-embedding-004</code> , was no longer supported in the current Gemini API environment	Updated the implementation to use <code>models/gemini-embedding-2</code>
The Gemini-based FAISS vector store could not be serialized using the tutorial's original pickle approach	Switched to FAISS's built-in <code>save_local()</code> / <code>load_local()</code> methods
The application could attempt to build an index from empty or missing URL fields	Added validation to check for empty URLs before attempting to build the FAISS index
The original tutorial's LLM (OpenAI) needed to be replaced	Adapted the LLM and embedding calls to Google Gemini throughout the pipeline

21. Security Considerations

The Gemini API key is stored separately in a local `secret_key.py` file rather than being hardcoded into the application logic. This file is excluded from version control via `.gitignore` and is not committed to GitHub.

22. Project Structure

```
PROJECT_1/
|
├─ main.py           # Streamlit app + RAG pipeline logic
├─ secret_key.py     # Local-only, holds the Gemini API key (not committed)
├─ requirements.txt  # Project dependencies
```

```
├── vector_index/
│   ├── index.faiss      # FAISS vector index
│   └── index.pkl        # FAISS index metadata
└── README.md
```

`secret_key.py` is sensitive local configuration and must not be pushed to a public repository.

23. Limitations

These reflect the difference between a portfolio implementation and a production-scale system, not defects in the current build:

- Local FAISS vector store (no cloud-hosted vector database)
- No user authentication
- No multi-user architecture
- No production monitoring or logging
- No automated RAG evaluation framework
- URL-based ingestion only (no file/document upload)
- No advanced hybrid (keyword + vector) search
- No reranking layer
- No cloud vector database

24. Future Enhancements

Enhancement	Status
PDF / document upload	Not implemented — future
Hybrid keyword + vector search	Not implemented — future
Reranking	Not implemented — future
Metadata filtering	Not implemented — future
Conversation memory	Not implemented — future
RAG evaluation (retrieval relevance)	Not implemented — future
Hallucination / groundedness evaluation	Not implemented — future
Cloud vector database	Not implemented — future
Authentication	Not implemented — future
Logging and monitoring	Not implemented — future
Docker deployment	Not implemented — future
Cloud deployment	Not implemented — future
Automated document refresh	Not implemented — future
Multiple document collections	Not implemented — future

25. Industry Relevance

This project demonstrates practical, hands-on understanding of:

- Generative AI and Large Language Models
- Retrieval-Augmented Generation (RAG)
- Semantic search and vector embeddings
- Vector databases and similarity search
- LangChain-based orchestration
- Prompt and context grounding
- Streamlit application development
- API integration

It should be understood as a **portfolio-level implementation of an industry-standard RAG architecture** — not as a production-grade or production-deployed system.

26. Skills Demonstrated

--	--

Skill	Demonstrated Through Project
Python	Core application logic and pipeline implementation
LangChain	Orchestration via <code>RetrievalQAWithSourcesChain</code>
Google Gemini	LLM used for answer generation (<code>gemini-3.5-flash-lite</code>)
Gemini Embeddings	Semantic vector representation (<code>models/gemini-embedding-2</code>)
RAG	End-to-end retrieval-augmented generation pipeline
FAISS	Vector index construction, persistence, and similarity search
Vector Search	Retrieval of relevant chunks via embedding similarity
Semantic Search	Meaning-based retrieval vs. keyword matching
Streamlit	Interactive user interface
NLP	Text extraction, chunking, and embedding of unstructured text
API Integration	Google Gemini API integration and key management
Document Processing	URL-based article loading and text splitting

27. Conclusion

RockyBot demonstrates a complete, working RAG pipeline: article ingestion, chunking, Gemini-based embedding, FAISS-based retrieval, and Gemini-based grounded answer generation, wrapped in a functional Streamlit interface. The implementation required adapting an OpenAI-based tutorial to Google Gemini, which surfaced and resolved real engineering issues around embedding model compatibility and vector store persistence. As a portfolio project, it demonstrates practical, applied understanding of the RAG pattern and its component technologies, while honestly distinguishing what has been implemented from what would be required to take the system to production scale.